



IntVariable

🕒 Created time	June 20, 2026 1:06 PM
☰ Category	CommonVariables
👤 Last edited by	 R.A R.A
🕒 Last updated time	July 31, 2026 10:54 AM

```
//Yes: 2, 18, 2854, -9

//No: 1.2, -5.7

int age = 0;

age = 43;

int ageInTenYears = age + 10;

//Two billion +/-
//signed Int32
//unsigned - 4 Billion
//bit - 0 or 1
//byte - 1 byte or 8 bits - 00000000
//1, 11, 111

Console.WriteLine(age);
Console.WriteLine(ageInTenYears);

//don't divide ints - Console.WriteLine(age / 2);
```

1. משתנים מסוג `int` (מספרים שלמים - Integers)

```
int = integer
```

- **ייעוד:** שמירת מספרים שלמים בלבד.
- **ערכים תקינים:** 2, 18, 2854, -9
- **ערכים לא תקינים:** 1.2, -5.7 (נקודה עשרונית לא תעבוד ב-Integers).
- **ברירת מחדל:** אם רק נצהיר על המשתנה ולא נציב בו ערך למשל

```
int age;
```

- הערך ההתחלתי שלו יהיה 0.
- **פעולות מתמטיות:** סוג הנתון הזה מצוין לפעולות **חיבור**, **חסור** ו**כפל**.

2. הקצאת ערכים ו-Types נתונים

- **הצבה מספרית:** כשנכתוב

```
//age = "43";  
Console.WriteLine(age);  
age = 43;
```

לא נשים מירכאות ("") סביב המספר 43.

- **סכנת שגיאה:** אם נכתוב

```
age = "43";
```

נקבל שגיאה.

מירכאות מגדירות **String**, וב-C# סוגי הנתונים לא חוצים קווים – אי אפשר להכניס **String** לתוך משתנה המוגדר כ-**int**.

- **ה-Conversion:** הפעולה

```
Console.WriteLine(age.ToString());
```

לוקחת את הערך המספרי וממירה אותו ל-**String** עוטפת אותו במירכאות מאחורי הקלעים כדי שנוכל למשל להדפיס אותו לקונסולה.

3. גבולות וזיכרון ב-Integers

- ה-**Int32** (ברירת המחדל): נקרא טכנית מאחורי הקלעים

signed Int32

המילה **signed** אומרת שיש לו סימן פלוס או מינוס.

הוא יכול להכיל כ-2 מיליארד לכיוון החיובי וכ-2 מיליארד לכיוון השלילי.

זה מה שאנחנו צריכים ברוב המוחלט של המקרים.

- ה-**UInt32**: מספר ללא סימן **unsigned**.

מכיוון שאין לו צד שלילי, כל הקיבולת עוברת לצד החיובי והוא יכול להחזיק עד כ-4 מיליארד.

- ה-**Int64**: אופציה למספרים עצומים בגודלם, אבל לרוב אנחנו לא משתמשים בה כי היא גדולה ממה שצריך.

4. איך מחשבים סופרים? ב-Bits וב-Bytes

- ה-**Bit**: היחידה הבסיסית ביותר במחשב, יכולה להיות **0** או **1**.

זה מתפקד כמו שער: פתוח (0) או סגור (1).

- ה-**Byte**: אוסף של 8 ביטים למשל

00000000

כשכל אפס יכול להפוך ל-1.

- שיטת הספירה הבינארית:

◦ ביט אחד = שתי אפשרויות (**0** או **1**).

◦ 2 ביטים = 4 אפשרויות (**00** זה 0, **01** זה 1, **10** זה 2, **11** זה 3).

◦ 3 ביטים = 8 אפשרויות (**000** עד **111**, המייצגים 0 עד 7).

- חוק ההכפלה (קריטי להבנה): כל פעם שמוסיפים ביט אחד, כמות האפשרויות מוכפלת.

מפתחים מתחילים נוטים להתבלבל פה: 16 ביט זה לא גדול פי 2 מ-8 ביט.

ה-9 ביטים זה פי 2 מ-8 ביטים.

בגלל ההכפלה הזו, כשמדברים על

Int32

כלומר 32 ביטים, מדובר בכמות עצומה של מקומות לאחסון **ints**.

5. שינוי משתנים (Variable Manipulation)

אנחנו רוצים לדעת מה יהיה הגיל שלנו בעוד 10 שנים:

```
int ageInTenYears = age + 10;  
Console.WriteLine(ageInTenYears);
```

כך אנחנו עובדים עם מספרים שלמים (**Integers**), אנחנו יכולים לחבר, לחסר, להכפיל ולחלק אותם.

אנחנו לא משנים את המספר המקורי בסוג כזה של משוואה:

```
int ageInTenYears = age + 10;
```

אם אנחנו רוצים לשנות את המספר המקורי במשוואה כזו:

```
age = age + 10;
```

זה ישנה את המשתנה המקורי, על ידי לקיחת ערך התוצאה והכנסתו חזרה למשתנה המקורי, מה שישנה את הגיל ל-53.

אנחנו צריכים להיזהר עם חילוק מכיוון שאם נכתוב את זה כך:

```
Console.WriteLine(age / 2);
```

מכיוון ש-**int** לא יודע להתמודד עם שאריות עשרוניות, הוא פשוט חותך אותן, ולכן צריך להיזהר מאוד עם חילוק של **Integers**.

חיבור, חיסור וכפל עובדים נהדר עם מספרים שלמים. אנחנו צריכים להיות מאוד זהירים עם חילוק כי התוצאות יהיו שגויות.

